Azure DevOps & CI/CD

Part 1 (Azure DevOps Fundamentals)

Question 1: What is Azure DevOps?

Interviewer

What is Azure DevOps? Explain its services and how you've used it in your projects.

Answer

Azure DevOps is a suite of services that supports the complete software development lifecycle (SDLC), from planning to deployment.

Azure DevOps Services

Service	Purpose
Azure Boards	Agile planning, work items, sprint management
Azure Repos	Git repositories
Azure Pipelines	CI/CD automation
Azure Test Plans	Manual and exploratory testing
Azure Artifacts	Package management (NuGet, npm, Maven, etc.)

Architecture

Developer

|

▼

Azure Repos

|

▼

Build Pipeline (CI)

|

▼

Artifacts

|

▼

Release Pipeline (CD)

|

▼

Dev → QA → UAT → Production

Real Project Example

In a typical ASP.NET Core project:

- Source code stored in Azure Repos
- Pull Requests required for merges
- CI pipeline builds and runs unit tests
- Artifacts published
- CD pipeline deploys to Azure App Service
- Approvals required before Production deployment

Interview Tip

For senior candidates, describe **your role**, not just Azure DevOps features.

Example:

I created YAML pipelines, configured CI/CD, integrated SonarQube for code quality, deployed to Azure App Service using deployment slots, and automated infrastructure deployment with Bicep.

Question 2: Explain Continuous Integration (CI) and Continuous Deployment (CD)

This is one of the most frequently asked DevOps questions.

Continuous Integration (CI)

Developers frequently merge code into the main branch.

Each commit triggers:

- Build
- Restore packages
- Static analysis
- Unit tests

- Artifact creation

Flow:

Developer Commit

|

▼

Build

|

▼

Unit Tests

|

▼

Publish Artifact

Continuous Delivery

The application is always ready for deployment.

Deployment to production still requires manual approval.

Continuous Deployment

Every successful build is automatically deployed to production without manual intervention.

Comparison

CI	CD (Delivery)	CD (Deployment)
Build & Test	Ready for Release	Automatically Released
Code Quality	Manual Approval	Fully Automated

Interview Question

What is the advantage of CI/CD?

Strong Answer

- Faster releases
- Reduced manual effort
- Early bug detection

- Consistent deployments
- Reduced deployment failures
- Faster recovery from issues

Question 3: Explain Azure Repos

Azure Repos is Git-based source control.

Supports:

- Branches
- Pull Requests
- Merge Policies
- Code Reviews

Git Workflow

main

|

├— develop

|

├— feature/login

|

├— feature/payment

|

└— release/1.2

Interview Question

What branching strategy do you use?

Strong Answer

For enterprise applications, I generally use a **main branch for production**, feature branches for development, and release or hotfix branches when needed. The exact strategy (GitFlow, Trunk-Based Development, or a simplified workflow) depends on the team's release cadence and deployment model.

Question 4: Explain Pull Requests (PR)

Before merging code,

Developer creates

Pull Request

Reviewer checks

✓ Code Quality

✓ Security

✓ Naming Standards

✓ Performance

✓ Unit Tests

Benefits

- Peer Review
- Knowledge Sharing
- Better Code Quality
- Reduced Bugs

Interview Question

What do you check during Code Review?

Senior Answer

I review:

- Business logic correctness

- SOLID principles
- Security issues
- Exception handling
- Logging
- Async usage
- Performance
- Naming conventions
- Test coverage
- Readability and maintainability

Question 5: Explain Azure Pipelines

Azure Pipelines automate build and deployment.

Flow

Developer Commit

↓

Pipeline Trigger

↓

Restore Packages

↓

Build

↓

Run Tests

↓

Publish Artifact

↓

Deploy

Pipeline Types

Classic Pipeline

GUI-based

YAML Pipeline

Code-based

Preferred.

Sample YAML

```
trigger:
- main

pool:
  vmImage: windows-latest

steps:
- task: DotNetCoreCLI@2
  inputs:
    command: restore

- task: DotNetCoreCLI@2
```

```
  inputs:
    command: build

- task: DotNetCoreCLI@2
  inputs:
    command: test
```

Interview Question

Why YAML instead of Classic?

Strong Answer

YAML	Classic
Version controlled	UI-based
Easy review	Harder to review
Reusable	Limited reuse
Infrastructure as Code	Configuration stored in UI

Question 6: Explain Build Pipeline

The Build Pipeline performs:

Restore

↓

Compile

↓

Run Tests

↓

Code Analysis

↓

Publish Artifact

Typical Steps

1. Restore NuGet packages
2. Build solution
3. Execute unit tests
4. Run static analysis (optional)
5. Publish build artifacts

Interview Question

Why publish artifacts?

Artifacts ensure that the **same tested build** is promoted through Dev, QA, UAT, and Production, avoiding differences between environments.

Question 7: Explain Release Pipeline

Release Pipeline deploys the built artifact.

Artifact

↓

Development

↓

QA

↓

UAT

↓

Production

Deployment Tasks

- Deploy App Service
- Update Configuration
- Execute Database Scripts (with appropriate controls)
- Restart Application
- Perform Smoke Tests

Interview Question

Should you rebuild for Production?

Strong Answer

No.

The same validated artifact should be promoted across environments to ensure consistency and reduce deployment risk.

Question 8: Explain Build Artifacts

Artifact

↓

Compiled Application

Example

MyApi.zip

Pipeline

Build

↓

Artifact

↓

Deploy

Advantages

- Versioned
- Immutable
- Reusable
- Traceable

Question 9: Explain Variable Groups

Variable Groups store shared configuration.

Example

Connection String

API Key

Environment Name

Advantages

- Centralized management
- Reusable across pipelines
- Reduced duplication

Best Practice

Store secrets in **Azure Key Vault** and reference them from Azure DevOps rather than storing them directly in variable groups.

Question 10: Explain Azure DevOps Agent

Azure DevOps Agent executes pipeline tasks.

Types

Microsoft Hosted Agent

Managed by Microsoft.

Examples

windows-latest

ubuntu-latest

Self-Hosted Agent

Managed by your organization.

Used when:

- Access to private networks is required
- Custom software or SDKs are needed
- Specialized build environments are required

Interview Question

When would you use a Self-Hosted Agent?

Strong Answer

When the build or deployment requires access to internal resources (such as private databases or on-premises systems), specific software not available on hosted agents, or compliance with organizational policies.

⭐ **Scenario-Based Question**

Interviewer

A build is failing after a developer commits code.

How do you troubleshoot?

Strong Answer

I would:

1. Review the pipeline logs to identify the failing step.
2. Determine whether the issue is restore, build, test, or publish.
3. Reproduce the issue locally if needed.
4. Verify SDK versions and NuGet package compatibility.
5. Check recent code changes.
6. Confirm pipeline variables and secrets.
7. Fix the issue, create a new commit, and validate the pipeline.

Top 10 Rapid-Fire Questions

Q1. Difference between CI and CD?

CI automates build and testing. CD automates release preparation or deployment.

Q2. Why YAML?

Pipeline as Code, version-controlled, and reusable.

Q3. What is Azure Repos?

Git repository service.

Q4. What is an Artifact?

A packaged output from the build that is deployed to environments.

Q5. Why use Pull Requests?

Code review, collaboration, and quality assurance.

Q6. Why use Hosted Agents?

Microsoft manages the build infrastructure.

Q7. Why use Self-Hosted Agents?

Access to private resources or custom environments.

Q8. What is a Variable Group?

Shared pipeline variables.

Q9. Why use Build Pipelines?

To compile, test, and package applications consistently.

Q10. Why use Release Pipelines?

To automate deployment through multiple environments.

Module 6 Progress

We've covered:

- Azure DevOps
- Azure Boards
- Azure Repos
- Git Workflow
- Pull Requests
- CI/CD
- Azure Pipelines
- YAML
- Build Pipelines
- Release Pipelines
- Build Artifacts
- Variable Groups
- Azure DevOps Agents
- Troubleshooting Build Failures

Next (Module 6 – Part 2)

We'll cover advanced DevOps topics that are frequently asked in senior interviews:

1. Git Merge vs Rebase
2. Branching Strategies (GitFlow vs Trunk-Based)
3. Deployment Strategies (Blue-Green, Canary, Rolling)
4. Infrastructure as Code (Bicep, ARM, Terraform)
5. SonarQube Integration
6. Security Scanning
7. Docker Basics
8. Kubernetes Basics (AKS)
9. Production Deployment Scenarios
10. Azure DevOps Best Practices

Given your experience with **Bicep, Azure deployments, VMSS, and App Services**, I'll also include practical troubleshooting questions similar to real production issues that senior Azure developers are often asked.

Excellent. This is the **most important part of Azure DevOps** for **Senior .NET Developers (8–12+ years)**.

As a Senior Interviewer, these are the questions I use to distinguish **mid-level developers** from **Senior/Lead Developers**.

Module 6 – Azure DevOps & CI/CD

Part 2 (Advanced DevOps)

Question 11: Explain Git Merge vs Git Rebase

Interviewer

What is the difference between Merge and Rebase? Which one do you prefer?

Answer

Both Merge and Rebase integrate changes from one branch into another.

Merge

Example

main

|

A------B------C

\

feature D------E

After Merge

main

|

A------B------C----------M

\ /

feature D----------E

A Merge Commit (M) is created.

Advantages

✓ Preserves complete history

✓ Easy to understand

✓ Safe

Disadvantages

- Extra merge commits

Rebase

Rebase moves feature commits on top of latest main.

Before

main

A------B------C

feature

A------B------D------E

After Rebase

main

A------B------C

\

feature D'------E'

History becomes linear.

Advantages

✓ Clean history

✓ Easier debugging

✓ Better commit sequence

Disadvantages

- Rewrites commit history
- Avoid rebasing shared branches

Interview Question

Which do you use?

Senior Answer

For **shared branches**, I prefer **Merge** because it preserves history and avoids rewriting commits.

For **local feature branches**, I may use **Rebase** before creating a Pull Request to keep history clean.

Question 12

Explain Branching Strategies

Very Frequently Asked

Git Flow

main

↓

develop

↓

feature/*

↓

release/*

↓

hotfix/*

Advantages

✓ Suitable for multiple release cycles

✓ Stable production

Trunk-Based Development

main

↓

Small Feature Branches

↓

Merge Daily

Advantages

✓ Faster Delivery

✓ Continuous Deployment

Feature Branch Workflow

main

↓

feature/login

↓

Pull Request

↓

main

Interview Question

Which strategy is best?

Senior Answer

It depends on the team's release model:

- **GitFlow** works well for organizations with scheduled releases and maintenance branches.
- **Trunk-Based Development** is often preferred for teams practicing Continuous Deployment with frequent releases.

Question 13

Explain Deployment Strategies

One of the favorite interview questions.

Blue-Green Deployment

Architecture

Users

↓

Blue (Current)

↓

Green (New)

↓

Switch Traffic

Advantages

✓ Zero Downtime

✓ Easy Rollback

Canary Deployment

Users

↓

10%

↓

New Version

↓

90%

↓

Old Version

Monitor

↓

Increase gradually.

Advantages

✓ Lower Risk

✓ Early Detection

Rolling Deployment

Server 1

↓

Update

↓

Server 2

↓

Update

↓

Server 3

Advantages

✓ No Downtime

✓ Suitable for multiple server instances

Interview Question

Which deployment strategy do you recommend?

Senior Answer

- **Blue-Green** for web applications requiring quick rollback.
- **Canary** when validating a release with a small percentage of users.
- **Rolling** for distributed environments such as Kubernetes or VM Scale Sets.

The choice depends on business requirements, infrastructure, and acceptable risk.

Question 14

Explain Infrastructure as Code (IaC)

Very Important

IaC means managing infrastructure using code.

Instead of

Portal

↓

Click

↓

Click

↓

Click

Use

Bicep

↓

Deploy

↓

Azure

Benefits

✓ Version Control

✓ Repeatability

✓ Automation

✓ Consistency

✓ Easier Disaster Recovery

Azure Tools

- Bicep
- ARM Templates
- Terraform

Interview Question

Why Bicep instead of ARM?

Senior Answer

ARM	**Bicep**
JSON	Cleaner syntax

ARM	Bicep
Verbose	Easier to read
Harder to maintain	Better developer experience
Native Azure	Native Azure

Example

```
resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'mystorage'
  location: resourceGroup().location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
}
```

Question 15

Explain SonarQube Integration

SonarQube performs static code analysis.

Pipeline

Code

↓

Build

↓

SonarQube

↓

Quality Report

Checks

✓ Code Smells

✓ Bugs

✓ Security Hotspots

✓ Duplicates

✓ Coverage

Interview Question

Should a pipeline fail if Sonar Quality Gate fails?

Senior Answer

Yes, for critical applications, failing the pipeline on a Quality Gate failure helps prevent poor-quality or insecure code from being deployed.

Question 16

Explain Secrets Management

Very Frequently Asked

Bad Practice

appsettings.json

↓

Password

Good Practice

Azure Key Vault

↓

Azure DevOps

↓

Pipeline

↓

Application

Best Practice

Never store

- Passwords
- API Keys
- Certificates

inside Git.

Question 17

Explain Docker

Docker packages an application and its dependencies into a container.

Architecture

ASP.NET Core

↓

Docker Image

↓

Container

↓

Azure Container Apps / AKS / App Service

Advantages

✓ Consistent environments

✓ Fast deployment

✓ Isolation

Dockerfile Example

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0

WORKDIR /app

COPY . .

ENTRYPOINT ["dotnet","MyApi.dll"]
```

Interview Question

Difference between Image and Container?

Image	Container
Blueprint	Running instance
Read-only	Executable

Question 18

Explain Kubernetes (AKS)

Kubernetes manages containers.

Architecture

Users

↓

Ingress

↓

AKS Cluster

↓

Pods

↓

Containers

Benefits

✓ Auto Scaling

✓ Self Healing

✓ Rolling Updates

✓ Load Balancing

Interview Question

Difference between Docker and Kubernetes?

Senior Answer

Docker creates and runs containers.

Kubernetes orchestrates and manages containers across multiple machines.

Question 19

Explain Production Deployment Process

Typical Enterprise Flow

Developer

↓

Pull Request

↓

Build

↓

Unit Tests

↓

SonarQube

↓

Artifact

↓

Dev

↓

QA

↓

UAT

↓

Approval

↓

Production

Production Checks

✓ Smoke Tests

✓ Health Checks

✓ Rollback Plan

✓ Monitoring

✓ Alerts

Question 20

Azure DevOps Best Practices

✓ YAML Pipelines

✓ Small Commits

✓ Pull Requests

✓ Branch Policies

✓ Code Reviews

✓ Automated Testing

✓ Infrastructure as Code

✓ Secrets in Key Vault

✓ Environment Approvals

✓ Monitoring after Deployment

⭐ **Scenario 1**

Interviewer

Production deployment failed.

What do you do?

Strong Senior Answer

1. Stop further deployments.
2. Review deployment logs to identify the failing step.
3. Determine whether the issue is infrastructure, configuration, application, or database related.
4. If necessary, roll back using the previous validated artifact or deployment slot.
5. Validate application health.
6. Perform root cause analysis.
7. Implement a permanent fix and document the incident.

⭐ **Scenario 2**

Interviewer

A deployment succeeded, but the application throws HTTP 500.

How do you troubleshoot?

Strong Answer

I would check:

- Application Insights exceptions

- Azure App Service logs
- Configuration and environment variables
- Azure Key Vault access
- Managed Identity permissions
- Database connectivity
- Startup logs
- Recent configuration changes
- Health Check endpoint
- Deployment slot configuration differences

⭐ **Scenario 3**

Interviewer

How would you deploy a .NET API with zero downtime?

Strong Senior Answer

I would use **Deployment Slots** in Azure App Service:

1. Deploy the new version to the **Staging** slot.
2. Execute smoke tests and health checks.
3. Validate database connectivity and dependencies.
4. Swap the Staging slot with Production.
5. Monitor telemetry (Application Insights, Azure Monitor).
6. If issues are detected, perform an immediate slot swap back (rollback).

This approach minimizes downtime and reduces deployment risk.

Top 15 Rapid-Fire Azure DevOps Questions

Q1. What is CI?

Automatic build and testing after code changes.

Q2. What is CD?

Automated delivery or deployment of validated artifacts.

Q3. Why YAML?

Version-controlled Pipeline as Code.

Q4. What is GitFlow?

A branching model using feature, release, and hotfix branches.

Q5. Merge vs Rebase?

Merge preserves history; Rebase creates a cleaner, linear history by rewriting commits.

Q6. Blue-Green Deployment?

Two identical environments with traffic switched between them.

Q7. Canary Deployment?

Gradually expose a new version to a small percentage of users.

Q8. Why Bicep?

Cleaner, more maintainable Infrastructure as Code for Azure.

Q9. What is SonarQube?

Static code analysis and quality gate enforcement.

Q10. Why use Key Vault?

Secure storage for secrets, keys, and certificates.

Q11. Docker vs Kubernetes?

Docker runs containers; Kubernetes orchestrates them.

Q12. Why use Deployment Slots?

Zero-downtime deployments and easy rollback.

Q13. What is an Artifact?

The packaged build output promoted through environments.

Q14. Why use Pull Requests?

Code quality, collaboration, and governance.

Q15. What is Infrastructure as Code?

Managing infrastructure using version-controlled code instead of manual configuration.

Module 6 Completed

You now have comprehensive coverage of:

- Azure DevOps Services
- Azure Repos
- Git
- Merge vs Rebase
- Branching Strategies
- CI/CD
- Azure Pipelines
- YAML Pipelines
- Build & Release Pipelines
- Build Artifacts
- Deployment Strategies
- Infrastructure as Code (Bicep/ARM/Terraform)
- SonarQube
- Secrets Management
- Docker
- Kubernetes (AKS)
- Production Deployments
- Zero-Downtime Deployments
- Troubleshooting Production Failures
- Senior-Level Scenario Questions

Next Module Recommendation: SQL Server & PostgreSQL (Senior Level)

This module will cover around **50+ senior interview questions**, including:

- SQL Query Optimization
- Clustered vs Non-Clustered Indexes
- Covering Indexes
- Execution Plans
- Stored Procedures
- Functions
- Views
- CTEs
- Window Functions (ROW_NUMBER, RANK, DENSE_RANK)
- Transactions
- ACID Properties
- Isolation Levels
- Deadlocks
- Locking & Blocking
- Query Performance Tuning
- Partitioning
- SQL Server vs PostgreSQL
- Real production troubleshooting scenarios

These topics are asked in almost every senior .NET interview because backend performance is a key responsibility for experienced developers.